

## Part 2)

Code before questions:

```
import psycopg
from getpass import getpass

connection = psycopg.connect(
    dbname="gradcafe",
    user="postgres",
    host="localhost",
    port=5432,
    password=getpass("PostgreSQL password: "),
)
```

Question 1) How many entries are for Fall 2026?

```
with connection.cursor() as cur:
    #Q1
    cur.execute("""
        SELECT COUNT(term) FROM applicants
        WHERE term ILIKE 'Fall 2026'

        """)
    print(f"Fall 2026 applicant count: {cur.fetchone()[0]}")
```

Result: Fall 2026 applicant count: 29577

Result after Pull Data and Update Analysis: 29,582

The WHERE condition selects Fall 2026 entries. COUNT(term) counts the matching entries.

Question 2) Among entries that provide a nationality classification, what percentage are international?

```
#Q2
cur.execute("""
    SELECT
        100.0 * COUNT(*) FILTER (
            WHERE us_or_international ILIKE 'International'
        )
        / NULLIF(COUNT(*), 0)
```

```

        FROM applicants
        WHERE us_or_international IS NOT NULL
              AND TRIM(us_or_international) <> ''
        """
    )

    print(f"Percent international: {cur.fetchone()[0]:.2f}%")

```

Result: Percent international: 46.33%

Result after Pull Data and Update Analysis: 46.32%

The WHERE conditions remove missing and blank classifications. The numerator counts international entries. The denominator counts every entry with a usable classification, including American and Other.

Question 3) What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of entries that provide each score?

#Q3

```

#GPA
cur.execute("""
    SELECT avg(gpa) from applicants
""")
print(f"Average GPA: {cur.fetchone()[0]:.2f}")

#GRE Quantitative
cur.execute("""
    SELECT avg(gre) from applicants
""")
print(f"Average GRE Quantitative: {cur.fetchone()[0]:.2f}")

#GRE Verbal
cur.execute("""
    SELECT avg(gre_v) from applicants
""")
print(f"Average GRE Verbal: {cur.fetchone()[0]:.2f}")

#GRE Analytical Writing
cur.execute("""
    SELECT avg(gre_aw) from applicants
""")
print(f"Average GRE Analytical Writing: {cur.fetchone()[0]:.2f}")

```

Result: Average GPA: 3.79

Average GRE Quantitative: 259.90

Average GRE Verbal: 161.53

Average GRE Analytical Writing: 8.34

Each AVG uses entries that provide that particular score. A missing GPA does not prevent an entry from contributing to a GRE average, for example. The unusual GRE averages reflect values in the submitted data; the queries do not correct or exclude them.

Question 4) What is the average GPA of American entries for Fall 2026?

#Q4

```
cur.execute("""
SELECT avg(gpa) FROM applicants
WHERE us_or_international ILIKE 'American'
AND term ILIKE 'Fall 2026'
""")
print(f"Average GPA of Americans for Fall 2026: {cur.fetchone()[0]:.2f}")
```

Result: Average GPA of Americans for Fall 2026: 3.79

The conditions select American Fall 2026 entries. AVG(gpa) uses those that provide a GPA and ignores missing GPA values.

Question 5) What percentage of Fall 2025 entries report an acceptance?

#Q5

```
cur.execute("""
SELECT
    100.0 * COUNT(*) FILTER (
        WHERE status ILIKE 'Accepted%'
    )
    / NULLIF(COUNT(*), 0)
FROM applicants
WHERE term ILIKE 'Fall 2025'
```

```

"""
print(f"Fall 2025 acceptance percentage: {cur.fetchone()[0]:.2f}%")

```

Result: Fall 2025 acceptance percentage: 47.92%

The numerator counts Fall 2025 entries whose status starts with Accepted. The denominator includes all Fall 2025 entries, including entries with another or missing status.

Question 6) What is the average GPA of accepted Fall 2026 entries?

```

#Q6
cur.execute("""
    SELECT avg(gpa) FROM applicants
    WHERE term ILIKE 'FALL 2026'
    AND status ILIKE 'Accepted%'
""")
print(f"Average GPA of accepted applicants in Fall 2026: {cur.fetchone()[0]:.2f}")

```

Result: Average GPA of accepted applicants in Fall 2026: 3.78

The conditions select accepted Fall 2026 entries. AVG(gpa) ignores entries without a GPA.

Question 7) How many entries are for a master's degree in Computer Science at Johns Hopkins University?

```

#Q7
cur.execute("""
    SELECT COUNT(*)
    FROM applicants
    WHERE (program ILIKE '%Johns Hopkins%'
    OR program ILIKE '%JHU%' )
    AND program ILIKE '%Computer Science%'
    AND degree ILIKE 'Masters'
""")
print(f"Applicants applied to Johns Hopkins University for a master's degree in Computer Science: {cur.fetchone()[0]}")

```

Result: Applicants applied to Johns Hopkins University for a master's degree in Computer Science: 8

This uses the original program and degree fields. An entry must match Johns Hopkins or JHU, Computer Science, and the Masters degree value.

Question 8) How many accepted Fall 2026 entries are for a Computer Science PhD at Georgetown, MIT, Stanford, or Carnegie Mellon, using the original fields?

```
#Q8
cur.execute("""
SELECT COUNT(*)
FROM applicants
WHERE (program ILIKE '%Georgetown University%'
OR program ILIKE '%Massachusetts Institute of Technology%'
OR program ILIKE '%, MIT%'
OR program ILIKE '%Stanford University%'
OR program ILIKE '%Carnegie Mellon University%')
AND program ILIKE '%Computer Science%'
AND degree ILIKE 'PhD'
AND term ILIKE 'FALL 2026'
AND status ILIKE 'Accepted%'
""")
q8_count = cur.fetchone()[0]
print(f"Accepted PhD in Computer Science applicants in Fall 2026 in Georgetown, MIT, Stanford and CMU is: {q8_count}")
```

Result: Accepted PhD in Computer Science applicants in Fall 2026 in Georgetown, MIT, Stanford and CMU is: 28

The query uses the original program field for the university and subject. It then requires a PhD, Fall 2026, and an accepted status. The Computer Science match also includes original program names such as Electrical Engineering and Computer Science.

Question 9) How does the Question 8 count change when the LLM-generated program and university fields are used?

#Q9

```
cur.execute("""
SELECT COUNT(*)
FROM applicants
WHERE (llm_generated_university ILIKE '%Georgetown University%'
OR llm_generated_university ILIKE '%Massachusetts Institute of Technology%'
OR llm_generated_university ILIKE 'MIT'
OR llm_generated_university ILIKE '%Stanford University%'
OR llm_generated_university ILIKE '%Carnegie Mellon University%')
AND llm_generated_program ILIKE '%Computer Science%'
AND degree ILIKE 'PhD'
AND term ILIKE 'FALL 2026'
AND status ILIKE 'Accepted%'
""")

q9_count=cur.fetchone()[0]
print(f"Original-field count: {q8_count}")
print(f"LLM-field count: {q9_count}")

cur.execute(
"SELECT %s::integer - %s::integer",
(q9_count, q8_count),
)
difference = cur.fetchone()[0]
print(f"Difference: {difference:+d}")
```

Result: Original-field count: 28

LLM-field count: 32

Difference: +4

This query uses `llm_generated_university` and `llm_generated_program` to identify the university and subject, while still using the original degree, term, and status. Four entries whose original program begins EECS have an LLM-generated program of Electrical Engineering and Computer Science. They therefore match Q9's Computer Science condition but not Q8's.

## PART 3)

Question 10) For Fall 2026 Computer Science applicants, whats the percentage of masters applicants and PHD applicants?

```
cur.execute("""
    SELECT
    COUNT(*) FILTER (WHERE degree ILIKE 'Masters') AS masters_count,
    COUNT(*) FILTER (WHERE degree ILIKE 'PhD') AS phd_count,
    COUNT(*) AS total_count,
    100.0 * COUNT(*) FILTER (WHERE degree ILIKE 'Masters')
    / NULLIF(COUNT(*), 0) AS masters_percentage,
    100.0 * COUNT(*) FILTER (WHERE degree ILIKE 'PhD')
    / NULLIF(COUNT(*), 0) AS phd_percentage
    FROM applicants
    WHERE term ILIKE 'Fall 2026'
    AND program ILIKE '%Computer Science%'
    """)

masters_count, phd_count, total_count, masters_percentage, phd_percentage =
cur.fetchone()
print(f"Computer Science entries in Fall 2026: {total_count}")
print(f"Masters: {masters_count} ({masters_percentage:.2f}%)")
print(f"PhD: {phd_count} ({phd_percentage:.2f}%)")
```

Result: Computer Science entries in Fall 2026: 1679

Masters: 569 (33.89%)

PhD: 1109 (66.05%)

The WHERE conditions select Fall 2026 entries whose original program contains Computer Science. COUNT(\*) gives the total of 1,679 entries. The FILTER conditions count the 569 master's entries and 1,109 PhD entries separately. Dividing each count by 1,679 gives 33.89% and 66.05%. The percentages do not add to 100% because one entry has its degree listed as Other.

Question 11) Among Fall 2026 Computer Science master's entries, how does the average reported GPA compare between accepted and rejected entries?

```
cur.execute("""
```

```

WITH averages AS (
SELECT
COUNT(gpa) FILTER (WHERE status ILIKE 'Accepted%') AS accepted_count,
AVG(gpa) FILTER (WHERE status ILIKE 'Accepted%') AS accepted_avg,
COUNT(gpa) FILTER (WHERE status ILIKE 'Rejected%') AS rejected_count,
AVG(gpa) FILTER (WHERE status ILIKE 'Rejected%') AS rejected_avg
FROM applicants
WHERE term ILIKE 'Fall 2026'
AND program ILIKE '%Computer Science%'
AND degree ILIKE 'Masters'
)
SELECT
accepted_count,
accepted_avg,
rejected_count,
rejected_avg,
accepted_avg - rejected_avg
FROM averages
"""
)

accepted_count, accepted_avg, rejected_count, rejected_avg, difference =
cur.fetchone()
print(f"Accepted: {accepted_count} reported GPAs, average {accepted_avg:.2f}")
print(f"Rejected: {rejected_count} reported GPAs, average {rejected_avg:.2f}")
print(f"Average GPA difference: {difference:+.2f}")

```

Result: Accepted: 280 reported GPAs, average 3.85

Rejected: 122 reported GPAs, average 3.74

Average GPA difference: +0.11

The query selects Fall 2026 Computer Science master's entries, then calculates separate GPA counts and averages for accepted and rejected statuses. Count (GPA) and avg (GPA) use only entries that provided a GPA. The final subtraction compares the two averages. This difference describes the submitted entries; it does not show that GPA caused the admission outcomes.